Task 1: Runtime Variables and Lifecycle Rules

1. Objective

To understand how Terraform input variables can be passed at runtime and how lifecycle rules control resource creation, updates, and destruction. The task uses Terraform's `local_file` and `random_pet` resources without creating any AWS infrastructure.

2. Steps Used

Step 1: Configure Terraform Resources

- Created the `main.tf` configuration file.
- Defined the `local_file.asif_pet` resource to create a text file.
- Defined the `random_pet.brownie` resource to generate a random pet name.
- Configured input variables for the filename and content.

Step 2: Execute Runtime Variables

Executed the following command in PowerShell:

```
terraform apply -var="filename=asif-runtime.txt" -var="content=I
believe in myself and keep moving forward"
```

- Approved the execution by entering `yes`.
- Verified the generated file using `Get-Content .\asif-runtime.txt`.
- Confirmed that the file contained the specified runtime content.

Step 3: Implement `create_before_destroy`

Added the following lifecycle configuration:

```
lifecycle {
  create_before_destroy = true
}
```

- Executed `terraform plan` to inspect the planned replacement.
- Observed the `+/-` action symbol, indicating creation before destruction.

- Executed `terraform apply` to test the lifecycle rule.
- Observed Terraform create the replacement resource before destroying the previous instance.
- Identified that using the same filename for both generations caused the resulting file to be removed during cleanup.

Step 4: Implement `ignore_changes`

Updated the lifecycle configuration:

```
lifecycle {
  ignore_changes = [content]
}
```

- Applied a baseline configuration with the content `Original content for Asif`.
- Changed the runtime content to `Updated content for Asif`.
- Executed:

```
terraform plan -var="filename=asif-runtime.txt" -var="content=Updated content for Asif"
```

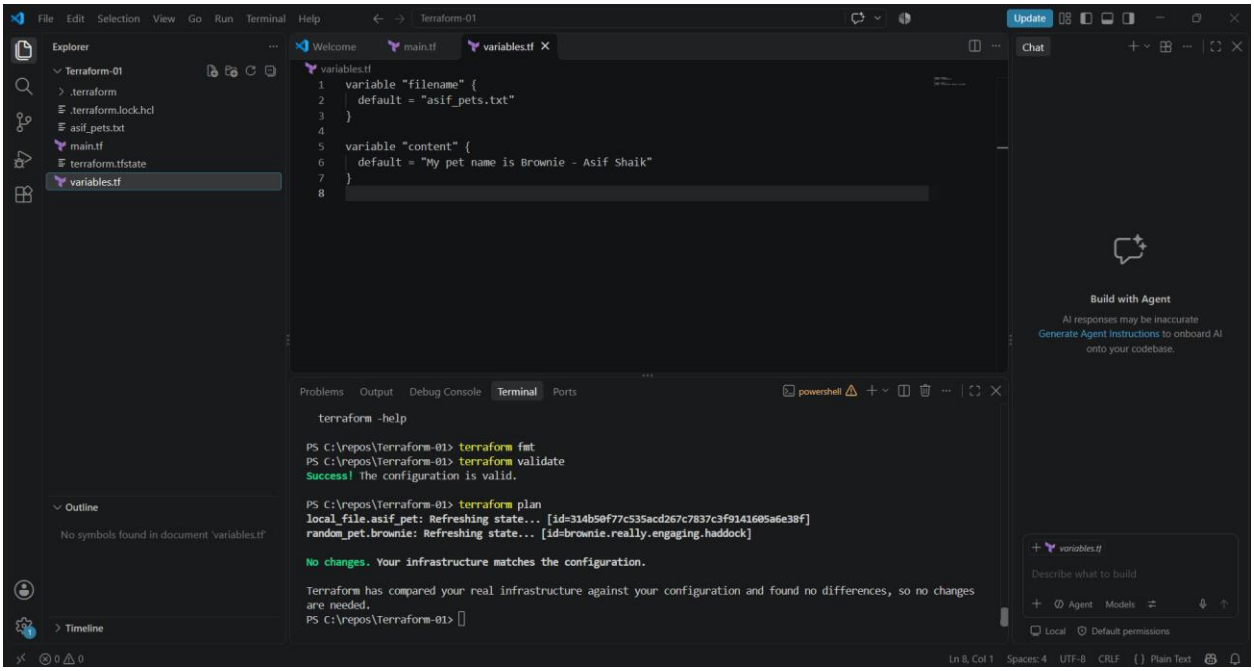
- Observed the message `No changes. Your infrastructure matches the configuration.`
- Verified that the file still contained `Original content for Asif`.

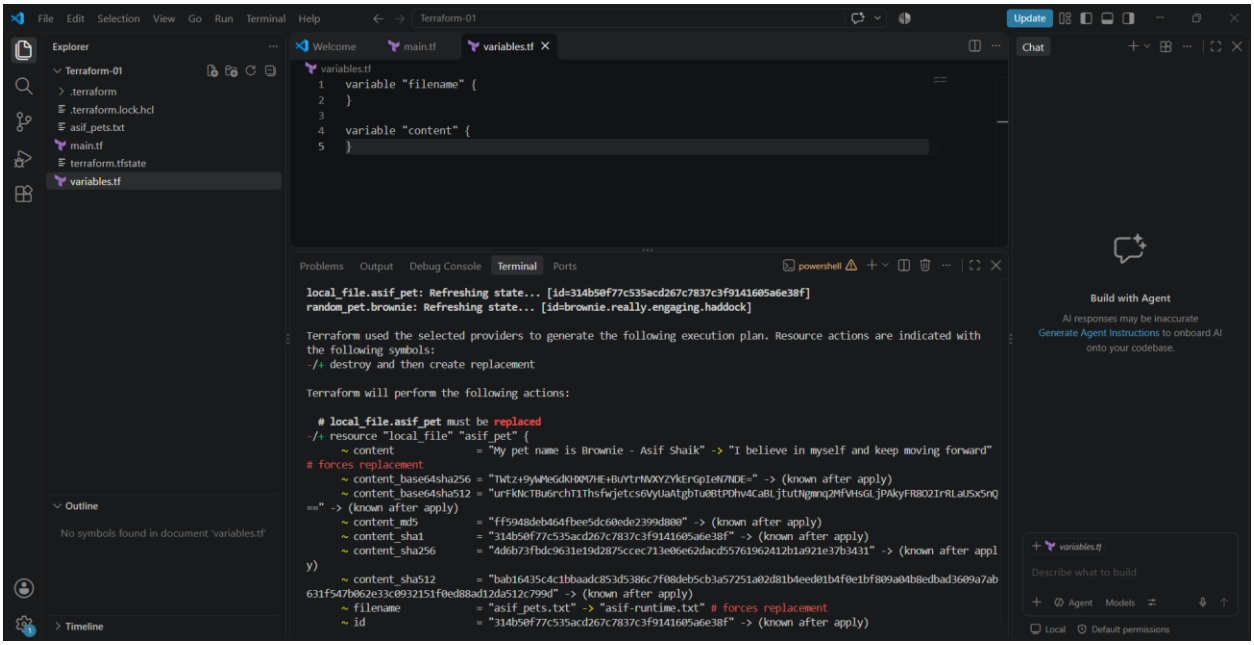
Step 5: Implement `prevent_destroy`

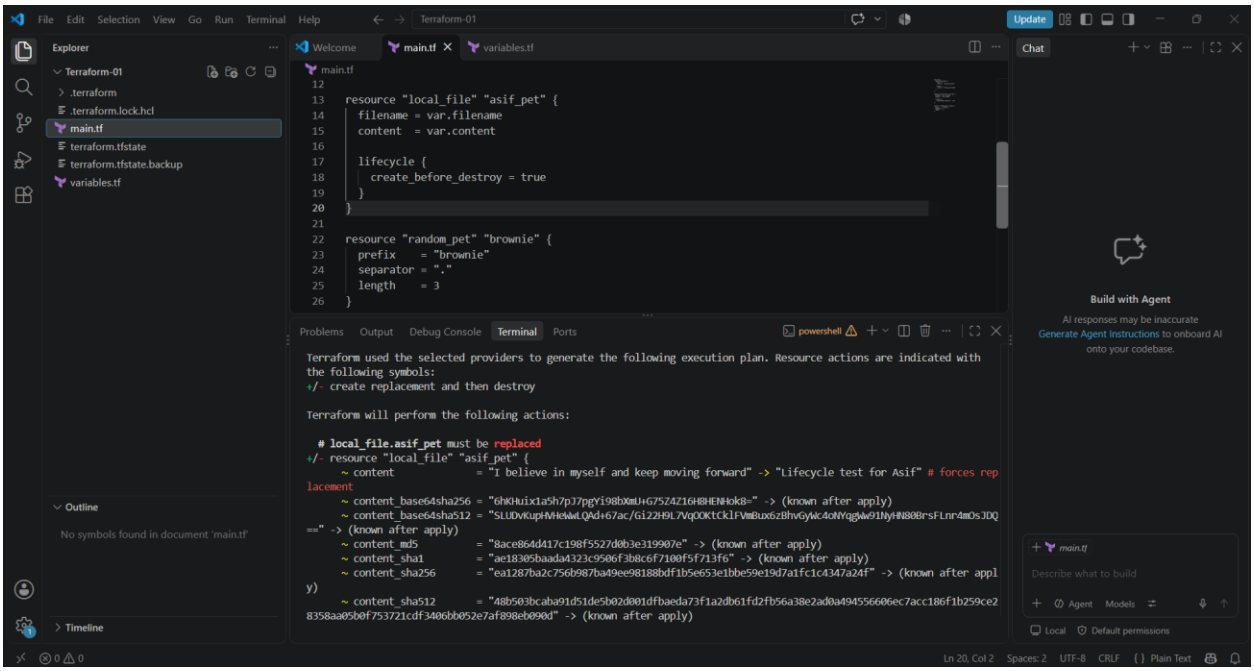
Updated the lifecycle configuration:

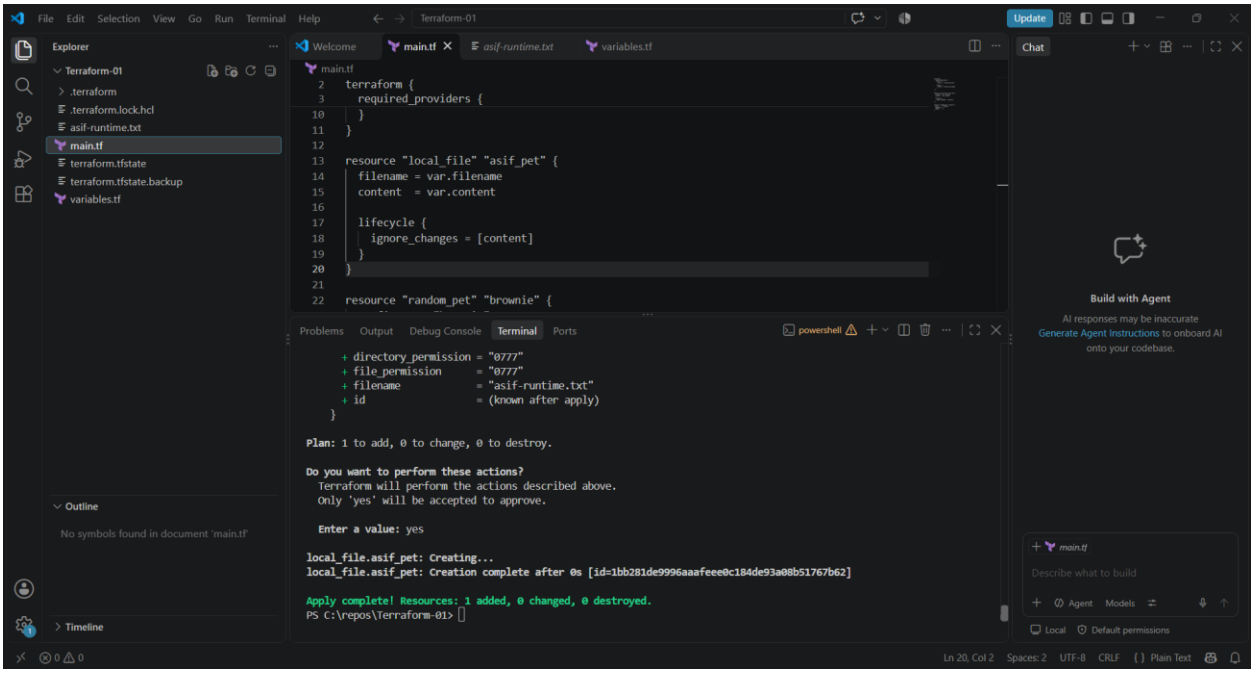
```
lifecycle {
  prevent_destroy = true
}
```

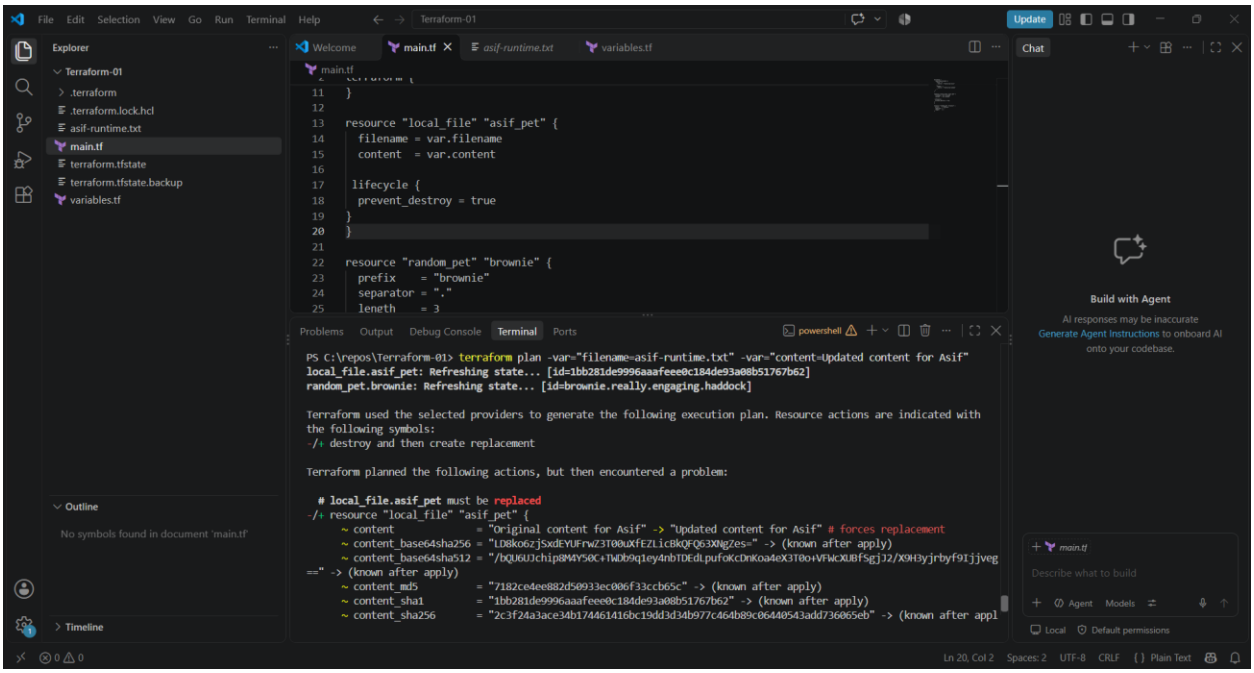
- Executed the Terraform plan with updated runtime content.
- Terraform attempted to replace the file because changing its content forces replacement.
- Observed the error `Instance cannot be destroyed`.
- Confirmed that Terraform blocked the replacement because `prevent_destroy` was enabled.


File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif_pets.txt
main.tf
terraform.tfstate
variables.tf
Welcome
main.tf
variables.tf
variables.tf
1 variable "filename" {
2 default = "asif_pets.txt"
3 }
4
5 variable "content" {
6 default = "My pet name is Brownie - Asif Shaik"
7 }
8
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
powershell
terraform -help
PS C:\repos\Terraform-01> terraform fmt
PS C:\repos\Terraform-01> terraform validate
Success! The configuration is valid.
PS C:\repos\Terraform-01> terraform plan
local_file.asif_pet: Refreshing state... [id=314b50f77c535acd267c7837c3f9141605a6e38f]
random_pet.brownie: Refreshing state... [id=brownie.really.engaging.haddock]
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\repos\Terraform-01>
Outline
No symbols found in document 'variables.tf'
Timeline
variables.tf
Describe what to build
Agent
Models
Local
Default permissions
Ln 8, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text


File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif_pets.txt
main.tf
terraform.tfstate
variables.tf
Welcome
main.tf
variables.tf
variables.tf
1 variable "filename" {
2 }
3
4 variable "content" {
5 }
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
powershell
local_file.asif_pet: Refreshing state... [id=314b50f77c535acd267c7837c3f9141605a6e38f]
random_pet.brownie: Refreshing state... [id=brownie.really.engaging.haddock]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
local_file.asif_pet must be replaced
-/+ resource "local_file" "asif_pet" {
~ content = "My pet name is Brownie - Asif Shaik" -> "I believe in myself and keep moving forward"
forces replacement
~ content_base64sha256 = "TWtz+9yWMeGdKH0M7HE+BuYtrNVXYZYkErGpIeN7NDE=" -> (known after apply)
~ content_base64sha512 = "urFkNcTBU6rchT1ThsfwjetcsGVyUaAtgbTu0BtPDhv4CaBLjtutNgmnq2MfVHsGLjPAkyFR8O2IrRLaUSx5nQ==" -> (known after apply)
~ content_md5 = "ff5948deb464fbee5dc60ede2399d800" -> (known after apply)
~ content_sha1 = "314b50f77c535acd267c7837c3f9141605a6e38f" -> (known after apply)
~ content_sha256 = "4d6b73fbdc9631e19d2875ccec713e06e62dacd55761962412b1a921e37b3431" -> (known after apply)
~ content_sha512 = "bab16435c4c1bbaadc853d5386c7f08deb5cb3a57251a02d81b4eed01b4f0e1bf809a04b8edbad3609a7ab631f547b062e33c0932151f0ed88ad12da512c799d" -> (known after apply)
~ filename = "asif_pets.txt" -> "asif-runtime.txt" # forces replacement
~ id = "314b50f77c535acd267c7837c3f9141605a6e38f" -> (known after apply)
Outline
No symbols found in document 'variables.tf'
Timeline
variables.tf
Describe what to build
Agent
Models
Local
Default permissions


File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf
Welcome
main.tf
variables.tf
Chat
main.tf
12
13 resource "local_file" "asif_pet" {
14 filename = var.filename
15 content = var.content
16
17 lifecycle {
18 create_before_destroy = true
19 }
20 }
21
22 resource "random_pet" "brownie" {
23 prefix = "brownie"
24 separator = "."
25 length = 3
26 }
Problems Output Debug Console Terminal Ports
powershell
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+/- create replacement and then destroy
Terraform will perform the following actions:
local_file.asif_pet must be replaced
+/- resource "local_file" "asif_pet" {
~ content = "I believe in myself and keep moving forward" -> "Lifecycle test for Asif" # forces replacement
~ content_base64sha256 = "6hKHuix1a5h7pJ7pgYi98bXmU+G75Z4Z16H8HENHok8=" -> (known after apply)
~ content_base64sha512 = "SLUDvKupHVHeWwLQAd+67ac/Gi22H9L7VqOOKtCklFVmBux6zBhvGyWc4oNYqgWw91NyHN80BrsFLnr4mOsJDQ==" -> (known after apply)
~ content_md5 = "8ace864d417c198f5527d0b3e319907e" -> (known after apply)
~ content_sha1 = "ae18305baada4323c9506f3b8c6f7100f5f713f6" -> (known after apply)
~ content_sha256 = "ea1287ba2c756b987ba49ee98188bdf1b5e653e1bbe59e19d7a1fc1c4347a24f" -> (known after apply)
~ content_sha512 = "48b503bcaba91d51de5b02d001dfbaeda73f1a2db61fd2fb56a38e2ad0a494556606ec7acc186f1b259ce28358aa05b0f753721cdf3406bb052e7af898eb090d" -> (known after apply)
Outline
No symbols found in document 'main.tf'
Timeline
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions
Ln 20, Col 2 Spaces: 2 UTF-8 CRLF { } Plain Text


File Edit Selection View Go Run Terminal Help
Terraform-01
Update
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
asif-runtime.txt
main.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf
Welcome
main.tf
asif-runtime.txt
variables.tf
Chat
main.tf
2 terraform {
3 required_providers {
10 }
11 }
12
13 resource "local_file" "asif_pet" {
14 filename = var.filename
15 content = var.content
16
17 lifecycle {
18 ignore_changes = [content]
19 }
20 }
21
22 resource "random_pet" "brownie" {
Problems Output Debug Console Terminal Ports
powershell
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "asif-runtime.txt"
+ id = (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.asif_pet: Creating...
local_file.asif_pet: Creation complete after 0s [id=1bb281de9996aaafeee0c184de93a08b51767b62]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\repos\Terraform-01>
Outline
No symbols found in document 'main.tf'
Timeline
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
main.tf
Describe what to build
Agent Models
Local Default permissions
Ln 20, Col 2 Spaces: 2 UTF-8 CRLF { } Plain Text



3. Observation

- Runtime variables allow filename and content values to be supplied through the command line.
- Terraform detects changes that require resource replacement.
- `create_before_destroy` changes the replacement order to create the new resource before destroying the old one.
- `ignore_changes = [content]` prevents Terraform from planning changes to the managed content attribute after creation.
- `prevent_destroy = true` blocks planned resource destruction and replacement while the lifecycle rule remains in the configuration.
- Terraform state continued to track both resources throughout the exercises.

4. Conclusion

The Terraform variables and lifecycle rules assignment was completed successfully. Runtime variable execution was verified, and all three lifecycle rules were tested through Terraform plans and applies.

The exercises demonstrated how Terraform manages configuration changes, controls replacement order, ignores selected attribute changes, and prevents accidental resource

destruction. The task was completed using local resources, without provisioning AWS infrastructure.

Task: Terraform Integration with Jenkins Pipeline

1. Objective

To integrate Terraform with Jenkins and automate Terraform operations through a Jenkins Pipeline, including configuration creation, initialization, validation, planning, applying, and verification.

2. Steps Used

1. **Verified Terraform Installation:** Confirmed Terraform was installed on the Jenkins EC2 server at `/usr/bin/terraform`, version 1.17.0.
2. **Configured Terraform in Jenkins:** Navigated to **Manage Jenkins → Tools → Terraform installations** and configured the installation name as `Terraform-Local`, with installation directory `/usr/bin` and automatic installation disabled.
3. **Created a Jenkins Pipeline Job:** Created a Pipeline job named `terraform-pipeline-lab`.
4. **Created Terraform Configuration:** Added a pipeline script that generates `main.tf` in the Jenkins workspace. The configuration defines two resources:
    a. `local_file.asif_pet` — creates `asif-runtime.txt` with the content `Original content for Asif`.
    b. `random_pet.brownie` — generates a random pet name using the prefix `brownie`.
5. **Initialized Terraform:** Executed `terraform init` to initialize the working directory and install the required providers.
6. **Validated the Configuration:** Executed `terraform validate` to check the Terraform configuration for errors.
7. **Generated the Execution Plan:** Executed `terraform plan` to preview the resources Terraform would create.
8. **Applied the Configuration:** Executed `terraform apply -auto-approve` to create the resources automatically without an interactive approval prompt.
9. **Verified the Output:** Checked the generated file content, displayed the random pet resource using `terraform state show`, and listed the generated workspace files.
10. **Verified Pipeline Status:** Checked the Jenkins Console Output and confirmed that the pipeline finished with `SUCCESS`.

3. Pipeline Code

```
pipeline {
    agent any

    stages {
        stage('Create Terraform Configuration') {
            steps {


                writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "asif_pet" {
  filename = "asif-runtime.txt"
  content  = "Original content for Asif"

  lifecycle {
    prevent_destroy = true
  }
}

resource "random_pet" "brownie" {
  prefix    = "brownie"
  separator = "."
  length    = 3
}
'''
            }
        }
```

```
stage('Terraform Init') {
    steps {
        sh '/usr/bin/terraform init'
    }
}

stage('Terraform Validate') {
    steps {
        sh '/usr/bin/terraform validate'
    }
}

stage('Terraform Plan') {
    steps {
        sh '/usr/bin/terraform plan'
    }
}

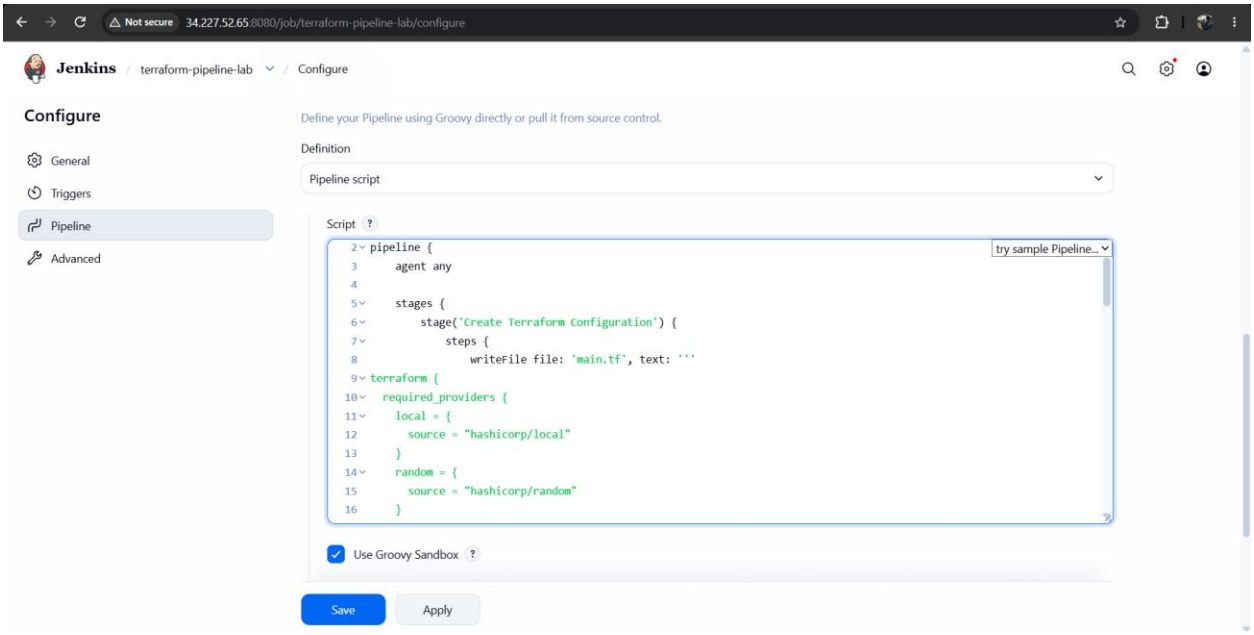
stage('Terraform Apply') {
    steps {
        sh '/usr/bin/terraform apply -auto-approve'
    }
}

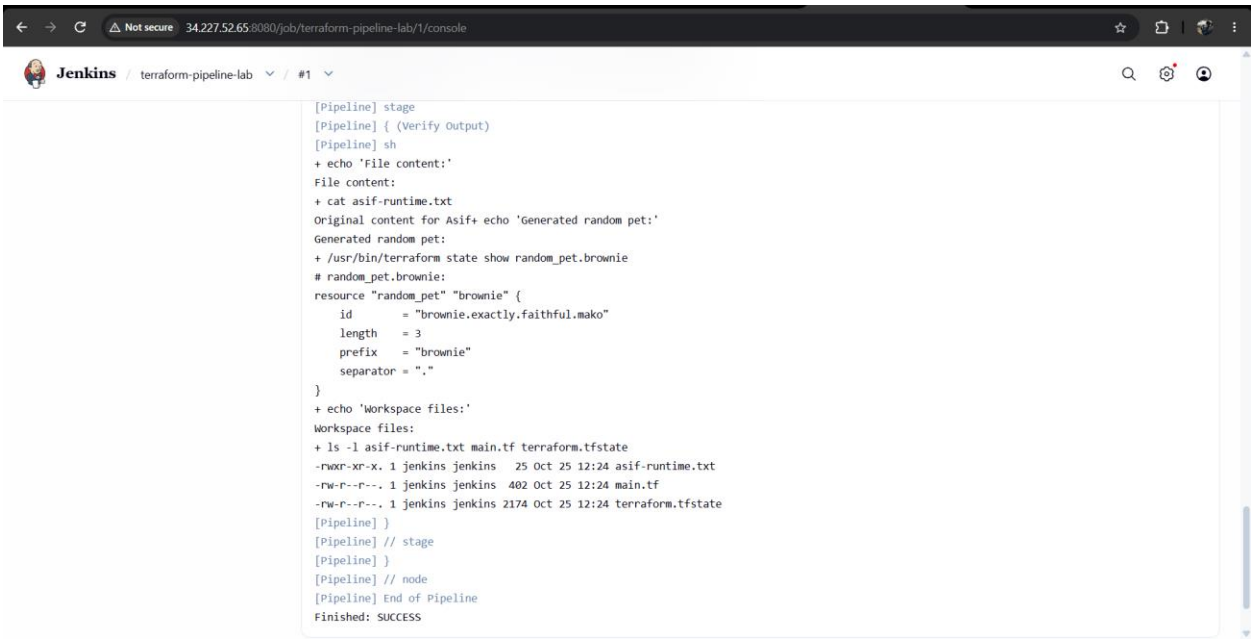
stage('Verify Output') {
    steps {
        sh '''
            echo "File content:"
            cat asif-runtime.txt

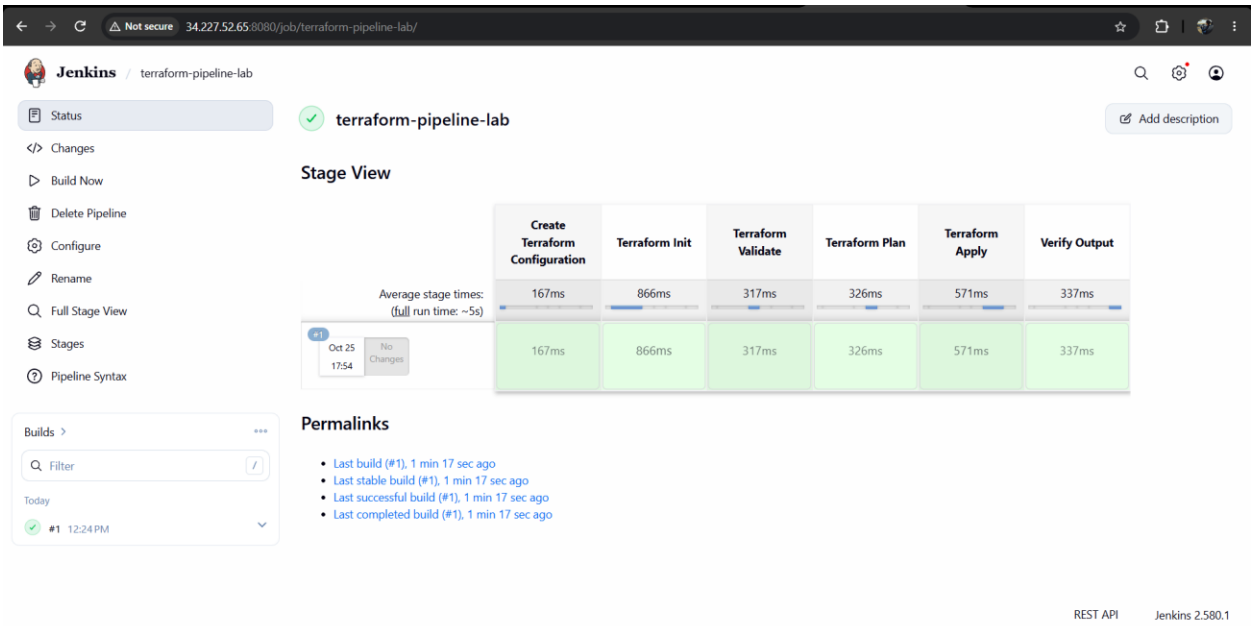
            echo "Generated random pet:"
            /usr/bin/terraform state show random_pet.brownie

            echo "Workspace files:"
            ls -l asif-runtime.txt main.tf terraform.tfstate
        '''
    }
}
```

```
    }
}
```


Not secure 34.227.52.65:8080/job/terraform-pipeline-lab/configure
Jenkins / terraform-pipeline-lab / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
stages {
stage('Create Terraform Configuration') {
steps {
writeFile file: 'main.tf', text: '''
terraform {
required_providers {
local = {
source = "hashicorp/local"
}
random = {
source = "hashicorp/random"
}
Use Groovy Sandbox
Save
Apply


Not secure 34.227.52.65:8080/job/terraform-pipeline-lab/1/console
Jenkins / terraform-pipeline-lab / #1
[Pipeline] stage
[Pipeline] { (Verify Output)
[Pipeline] sh
+ echo 'File content:'
File content:
+ cat asif-runtime.txt
Original content for Asif+ echo 'Generated random pet:'
Generated random pet:
+ /usr/bin/terraform state show random_pet.brownie
random_pet.brownie:
resource "random_pet" "brownie" {
id = "brownie.exactly.faithful.mako"
length = 3
prefix = "brownie"
separator = "."
}
+ echo 'Workspace files:'
Workspace files:
+ ls -l asif-runtime.txt main.tf terraform.tfstate
-rwxr-xr-x. 1 jenkins jenkins 25 Oct 25 12:24 asif-runtime.txt
-rw-r--r--. 1 jenkins jenkins 402 Oct 25 12:24 main.tf
-rw-r--r--. 1 jenkins jenkins 2174 Oct 25 12:24 terraform.tfstate
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS



4. Observation

- Terraform was successfully initialized, and the required `local` and `random` providers were installed.
- Terraform validation completed successfully.
- The execution plan showed **2 resources to add, 0 to change, and 0 to destroy**.
- Terraform created the local file `asif-runtime.txt` with the expected content.
- The random pet resource generated the ID `brownie.exactly.faithful.mako`.
- The Terraform state file, `terraform.tfstate`, was created to track the managed resources.
- All pipeline stages completed without errors, and Jenkins displayed **Finished: SUCCESS**.

5. Conclusion

Successfully integrated Terraform with Jenkins using a declarative Pipeline. Automated the Terraform initialization, validation, planning, application, and verification stages. This lab demonstrated how Jenkins can execute Terraform commands in a repeatable workflow and verify the resulting resources.